

DuckDB from the Command Line

A Second Way to Talk to Your Database — macOS

Table of Contents

1. Why Learn the Command Line?
2. Two Different "DuckDBs"
3. What You'll Need
4. Installing the DuckDB CLI on macOS
5. Apple Silicon vs. Intel — Why It Matters Here
6. Verifying the Install
7. Starting DuckDB
8. The DuckDB Prompt
9. Dot-Commands — Your Toolbox
10. Changing How Results Look
11. Querying a CSV — No Loading Step

Why Learn the Command Line?

So far, you've used DuckDB **inside Marimo notebooks**. That's great for building and sharing analysis.

The **command line** is different — it's a fast, no-frills way to:

- Quickly check a table without opening a notebook
- Run a `.sql` file as part of a script or pipeline
- Peek inside a `.csv` file in seconds
- Practice SQL the way many working analysts actually do

👉 Notebooks and the command line are **two tools for the same job**.

Knowing both makes you more flexible.

Two Different "DuckDBs"

You already have the **DuckDB Python package** installed (`pip install duckdb`). That's what powers `import duckdb` in Marimo.

Today we install a **second, separate thing**: the **DuckDB CLI** — a standalone program called `duckdb` that you run directly in **Terminal**, with no Python involved at all.

	Python package	DuckDB CLI
How you use it	<code>import duckdb</code> in a script/notebook	Type <code>duckdb</code> in Terminal
Needs Python?	Yes	No
Best for	Notebooks, analysis with pandas	Quick checks, scripts, learning SQL

What You'll Need

- The **Terminal** app (press `Cmd + Space`, type `Terminal`, press Enter)
- 5–10 minutes to install the DuckDB CLI (next slides)
- The sample file `data/students.csv` in this same tutorial folder
- **macOS 11 (Big Sur) or newer** — the DuckDB CLI's Homebrew formula does not support older versions

That's it — no server, no account, no configuration file.

Installing the DuckDB CLI — Option A: Homebrew (recommended)

Homebrew is the standard package manager for macOS.

Check whether you already have it:

```
brew --version
```

If that prints a version number, install DuckDB with one command:

```
brew install duckdb
```

If you see `zsh: command not found: brew`, install Homebrew first:

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Then follow the two `echo`/`eval` lines Homebrew prints at the end — they add `brew` itself to your PATH. Close and reopen Terminal, then

```
run brew install duckdb
```

Installing the DuckDB CLI — Option B: Manual Download

No Homebrew, or it's not letting you install right now? Get the binary directly:

1. Go to duckdb.org/install and choose **macOS + CLI**
2. Download the `.zip` for your Mac's chip (see next slide if you're not sure which one)
3. Double-click the `.zip` in **Downloads** to unzip it — you get a single file named `duckdb`
4. Move it into a folder already on your PATH and make it executable:

```
chmod +x ~/Downloads/duckdb
sudo mv ~/Downloads/duckdb /usr/local/bin/duckdb
```

Apple Silicon vs. Intel — Why It Matters Here

The manual download offers two different `.zip` files, and picking the wrong one gives a binary that won't run. Check which Mac you have:

```
uname -m
```

Output	Your Mac	Download labeled
<code>arm64</code>	Apple Silicon (M1/M2/M3/M4)	<code>osx_arm64</code>
<code>x86_64</code>	Intel	<code>osx_amd64</code>

👉 **Homebrew (Option A) handles this automatically** — it is the main reason this course recommends Homebrew over the manual download.

"Apple could not verify..." — Gatekeeper Warning

If you double-click the downloaded `duckdb` binary (or run it) and macOS refuses with a message about an **unidentified developer**, it's Gatekeeper — the manual download isn't notarized the way an App Store app is. Clear the warning from Terminal instead of clicking through dialogs:

```
xattr -d com.apple.quarantine ~/Downloads/duckdb
```

Then retry the `chmod +x` / `mv` steps from the previous slide. Homebrew installs never trigger this warning.

Verifying the Install

Close and reopen Terminal (so your shell picks up any PATH change), then run:

```
duckdb --version
```

You should see something like:

```
v1.1.3 19864453f7
```

Confirm **where** it's running from:

```
which duckdb
```

Homebrew on Apple Silicon prints `/opt/homebrew/bin/duckdb`; on an Intel Mac, `/usr/local/bin/duckdb`. Either is correct — they're just Homebrew's two different install locations.

Starting DuckDB — In-Memory

Just type `duckdb` with nothing after it:

```
duckdb
```

```
v1.1.3 19864453f7
Enter ".help" for usage hints.
Connected to a transient in-memory database.
Use ".open FILENAME" to reopen on a persistent database.
D
```

The `D` is your **prompt** — DuckDB is waiting for a command.

⚠ "In-memory" means: when you close this window, **everything you created is gone**. Good for quick experiments.

Starting DuckDB — Persistent File

To **save** your work to disk, give `duckdb` a filename:

```
duckdb my_class.db
```

- If `my_class.db` doesn't exist yet, DuckDB **creates** it.
- If it already exists, DuckDB **reopens** it — your tables are still there.

👉 This is exactly like the difference between

```
duckdb.connect(':memory:') and duckdb.connect('my_class.db')
```

in your Marimo notebooks.

An **absolute** Mac path always uses forward slashes and just works, as typed:

```
duckdb /Users/maria/Desktop/OMIS-105-LABS/my_class.db
```

The DuckDB Prompt

Once you see the `D` prompt, you can type SQL directly:

```
D SELECT 6 * 7 AS answer;
```

answer
int32
42

Press **Enter** after the `;` to run the statement.

Don't Forget the Semicolon

If you forget the `;`, DuckDB just waits for more input:

```
D SELECT 6 * 7 AS answer
...>
```

Type the `;` on the next line and press Enter to finish it.

⚠ This is the #1 "why isn't anything happening?" moment for beginners. When the prompt changes to `...>`, DuckDB is still listening — it just needs your semicolon.

Dot-Commands — Your Toolbox

Commands that start with a **dot (.)** are not SQL — they're DuckDB CLI shortcuts. No semicolon needed.

```
.help      show all dot-commands
.tables    list tables in the current database
.schema    show CREATE TABLE statements
.mode      change how results are displayed
.headers   show/hide column names
.timer     show how long each query took
.quit      exit the CLI
```

👉 Think of dot-commands as **settings for your session**, and SQL as **questions for your data**.

.tables and .schema

Create a table, add some rows, then inspect it:

```
D CREATE TABLE students (id INTEGER, name VARCHAR, gpa DOUBLE);
D INSERT INTO students VALUES
  (1, 'Alice', 3.8), (2, 'Bob', 3.5), (3, 'Charlie', 3.9),
  (4, 'Diana', 3.2), (5, 'Ethan', 2.9), (6, 'Fiona', 3.7),
  (7, 'George', 3.1), (8, 'Hana', 3.95);
D .tables
students
D .schema students
CREATE TABLE students(id INTEGER, name VARCHAR, gpa DOUBLE);
```

`.tables` answers *"what tables exist?"*

`.schema` answers *"what do they look like?"*

👉 The next few slides keep querying this same `students` table — it stays alive for the rest of your session.

.mode — Changing How Results Look

The default look (`box` , shown so far) is nice to read, but not easy to paste elsewhere. Switch styles with `.mode` :

```
D .mode csv
D SELECT * FROM students LIMIT 2;
id,name,gpa
1,Alice,3.8
2,Bob,3.5
```

Other useful modes: `.mode markdown` (great for pasting into a README), `.mode json`, `.mode line`. Switch back anytime with `.mode box`.

.headers and .timer

```
D .headers off
D SELECT name FROM students LIMIT 1;
Alice
```

```
D .timer on
D SELECT COUNT(*) FROM students;
```

count_star()
8

```
Run Time (s): real 0.001 user 0.000539 sys 0.000315
```

`.timer on` is handy once your tables get big and you want to know if a query is fast or slow.

Querying a CSV — No Loading Step

This is DuckDB's superpower, and it works the same on the command line as it does in a notebook. No `CREATE TABLE`, no import wizard — the file **is** the table.

First, get into this tutorial's folder — either `cd` there, or drag the folder onto Terminal after typing `cd` with a trailing space:

```
cd ~/Desktop/OMIS-105/tutorials/DuckDB_from_Command_Line
```

(Adjust the path if you cloned or unzipped the course repo somewhere else. Confirm with `pwd` and `ls data/`.)

Querying a CSV — Running the Query

```
D SELECT major, ROUND(AVG(gpa), 2) AS avg_gpa
FROM read_csv('data/students.csv')
GROUP BY major
ORDER BY avg_gpa DESC;
```

major	avg_gpa
Information Systems	3.83
Accounting	3.50
Marketing	3.35
Finance	3.35

'data/students.csv' is a **relative path** — it only resolves correctly if Terminal is standing in this tutorial's folder (see the previous slide).

One-Off Queries with `-c`

Sometimes you don't want to open the interactive prompt at all — you just want **one quick answer**. Run it straight from Terminal (not from inside `duckdb`) using `-c`:

```
duckdb -c "SELECT COUNT(*) FROM read_csv('data/students.csv')"
```

count_star()
8

👉 Great for quick checks without leaving your normal Terminal workflow.

Quoting on macOS — `zsh` Notes

macOS Terminal's default shell is `zsh`. Two rules keep `-c` queries safe:

- Wrap the **whole SQL statement** in **double quotes** (`"..."`), and use **single quotes** for any string literal *inside* the SQL (`'data/students.csv', WHERE major = 'Finance'`). Mixing them the other way around breaks the shell's parsing.
- `zsh` treats `!` inside a **double-quoted** string as a history trigger and may error with `zsh: event not found`. This won't come up in this course's examples, but if you ever add `!=` inside a `-c` string and hit that error, switch the outer quotes to single quotes and escape the inner ones instead.

Running a Script File

1. Create the file. Open any plain-text editor (VS Code, TextEdit in **plain text mode** — not rich text/Word), type the SQL below, and save it as `report.sql` in this same folder:

```
SELECT major, COUNT(*) AS n
FROM read_csv('data/students.csv')
GROUP BY major;
```

2. Run it. Back in Terminal (not inside the `duckdb` prompt):

```
duckdb -c ".read report.sql"
```

👉 `report.sql` is just text — DuckDB reads it and runs each statement in order. Save once, rerun anytime: that's **repeatable** SQL.

Leaving DuckDB

Any of these will exit the interactive prompt:

```
D .quit
```

```
D .exit
```

Or press **Ctrl+D** — the standard "end of input" shortcut on macOS and Linux terminals.

⚠ If you used an **in-memory** session, everything you built disappears the moment you exit. Use a file (`duckdb my_class.db`) if you want it to survive.

Common Mistakes

- **Forgetting the `;`** → prompt hangs at `...>`. Type `;` + Enter.
- **Typing a dot-command with a `;`** → dot-commands don't need one (`.tables;` will error).
- **Quoting strings incorrectly** → SQL text needs single quotes: `WHERE major = 'Finance'`, not double quotes.
- **Using an in-memory session, then closing Terminal** → your table is gone. Use a `.db` file if you need it saved.
- **Running `duckdb` from the wrong folder** → `read_csv('data/students.csv')` only works if Terminal is in this tutorial's folder. Check with `pwd`.
- **`zsh: command not found: duckdb` right after installing** → you didn't reopen Terminal, so your shell hasn't reloaded its PATH.

Troubleshooting

`zsh: command not found: duckdb`

1. Reopen Terminal completely (not just a new tab in some setups) — PATH changes from Homebrew only take effect in a fresh shell.

2. Confirm Homebrew's own PATH lines are in your shell profile:

`grep brew ~/.zprofile` should show an `eval "$(/opt/homebrew/bin/brew shellenv)"`

line (Apple Silicon) or `/usr/local/bin/brew shellenv` (Intel).

If missing, re-run the two lines `brew install`'s output showed you.

3. Still stuck? Run `brew list duckdb` to confirm it's actually installed, then `brew --prefix duckdb` to see exactly where.

Troubleshooting (continued)

"Apple could not verify this app is free of malware"

You downloaded the binary manually instead of via Homebrew. Run

`xattr -d com.apple.quarantine <path-to-duckdb>` (see the Gatekeeper slide earlier), or just switch to `brew install duckdb` — it never triggers this.

`duckdb: command not found` only inside VS Code's terminal

VS Code's integrated terminal sometimes starts before your shell profile changes take effect. Close and reopen the VS Code terminal panel, or open a plain Terminal window instead to confirm the install itself is fine.

My CSV query says "No files found that match the pattern"

Cheat Sheet

Command	What it does
<code>brew install duckdb</code>	install the DuckDB CLI (recommended)
<code>duckdb --version</code>	confirm the install worked
<code>which duckdb</code>	show which copy of <code>duckdb</code> will run
<code>duckdb</code>	start in-memory session
<code>duckdb file.db</code>	start/reopen a persistent database
<code>duckdb -c "SQL"</code>	run one query from Terminal, then exit
<code>duckdb -c ".read file.sql"</code>	run a whole script file
<code>.tables</code>	list tables
<code>.schema [table]</code>	show table structure
<code>.mode box csv markdown json</code>	change output style

Practice Exercise

In Terminal, `cd` into this tutorial's folder, then:

1. Confirm the install: `duckdb --version` and `which duckdb`
2. Start DuckDB: `duckdb`
3. Run: `SELECT * FROM read_csv('data/students.csv');`
4. Find the average GPA **per major**, highest first
5. Switch to `.mode markdown` and rerun your query
6. Save your query in a file called `my_query.sql`,
then run it with `duckdb -c ".read my_query.sql"`
7. Exit with `.quit` or **Ctrl+D**

👉 If you can do all seven steps, you can use DuckDB from the command line on your Mac with confidence.

You're Ready

Notebooks for building analysis.

The command line for quick answers and repeatable scripts.

Resources

duckdb.org/docs/api/cli — Official CLI documentation

duckdb.org/install — Install DuckDB for macOS (Homebrew or manual)

OMIS 105 — Introduction to Database Management Systems — Fall 2026